WorksheetDownload.com

10 Q&A ON BROKERS, TASK ROUTING, RETRIES, BEAT, AND TESTING · PRACTICE WORKSHEET

CONCEPTUAL SERIES

Celery

Name: ______________________ Cohort/Batch: ______________________ Date: ______________________ Score: ______

CELERY PRACTICE SHEET - 10 CONCEPTUAL Q&A

10 Q&A on brokers, task routing, retries, Beat, and testing

TOTAL: 10 QUESTIONS

Q1 What is Celery and what role does a message broker play? *Threat Model*

Q2 What is the difference between delay() and apply_async()? *Architecture*

Q3 How does Celery handle task retries and what is autoretry_for? *Configuration*

Q4 What is Celery Beat and how do you schedule a periodic task? *Batching*

Q5 What are Celery chains, groups, and chords? *SSO / IdP*

Q6 How do you route tasks to specific queues in Celery? *Observability*

Q7 How does Celery handle task result storage? *Lifecycle*

Q8 What is the Celery Flower monitoring dashboard? *Compliance*

Q9 How do you test Celery tasks without a running broker? *Testing*

Q10 What are Celery soft and hard time limits? *Tuning*

ANSWER KEY & EXPLANATORY SOLUTIONS

Comprehensive verified answers, best-practice configs, security controls & reference

VERIFIED SOLUTIONS

A1 Celery is a distributed task queue. When

Mitigation

Celery is a distributed task queue. When you call task.delay(), Celery serializes the call and sends it to a broker (Redis or RabbitMQ). Worker processes consume messages from the broker, execute tasks, and optionally store results in a backend.

A2 task.delay(arg1, arg2) is shorthand for apply_async((arg1,

Primitives

task.delay(arg1, arg2) is shorthand for apply_async((arg1, arg2)). apply_async gives you full control: countdown (delay in seconds), eta (absolute time), queue name, priority, retry policy, and task id override.

A3 Call self.retry(exc=e, countdown=5, max_retries=3) inside a

Hardening

Call self.retry(exc=e, countdown=5, max_retries=3) inside a task on failure. @app.task(autoretry_for=(RequestException,), retry_backoff=True, max_retries=5) automates this: Celery retries on the listed exceptions with exponential backoff.

A4 Celery Beat is a scheduler that enqueues

Gotchas

Celery Beat is a scheduler that enqueues tasks on a schedule. Define entries in beat_schedule: {'run-daily': {'task': 'app.tasks.cleanup', 'schedule': crontab(hour=0, minute=0)}}. Beat requires a persistent store (db or Redis) to track last-run times.

A5 A chain(t1.s(), t2.s()) pipes output of t1

Federation

A chain(t1.s(), t2.s()) pipes output of t1 as input to t2. A group([t1.s(), t2.s()]) runs tasks in parallel. A chord(group, callback) runs the group in parallel then passes all results to callback useful for map-reduce-style workflows.

A6 Define queues in the task_queues config. Set

Observability

Define queues in the task_queues config. Set task_routes = {'myapp.tasks.heavy': {'queue': 'heavy_queue'}}. Start workers with --queues heavy_queue to consume from that queue. This segregates slow tasks from fast ones for independent scaling.

A7 Set result_backend = 'redis://localhost' (or

Automation

Set result_backend = 'redis://localhost' (or 'db+postgresql://...'). After completion, AsyncResult(task_id).get() blocks until the result is available. Results are stored for result_expires seconds (default 24 h) then purged.

A8 Flower is a real-time web UI for

Governance

Flower is a real-time web UI for Celery. Run celery -A app flower. It displays active workers, task rates, queue lengths, failed task details, and lets you revoke pending tasks or rate-limit workers from the browser.

A9 Set task_always_eager = True in the test

Assessment

Set task_always_eager = True in the test config. This causes .delay() and .apply_async() to execute the task synchronously in the calling process without a broker. For unit tests of task logic only, call the function directly: mytask(args) skipping Celery entirely.

A10 soft_time_limit raises SoftTimeLimitExceeded inside the task,

Optimization

soft_time_limit raises SoftTimeLimitExceeded inside the task, giving it a chance to clean up. hard time_limit (task_time_limit) sends SIGKILL to the worker process if the task exceeds the limit, with no cleanup possible. Prefer soft limits.